Payment Gateway: LLD Notes

Scope

Category	Item	Note
In scope	Multiple payment methods	Card and UPI built; PayPal is an extension (new class + one map entry)
In scope	Routing to the right processor	Map keyed by method.type() in Gateway
In scope	Retry on failure	Single RetryProcessor decorator; retries only RetryableException
In scope	Status tracking	PENDING, SUCCESS, FAILED on Payment
In scope	Notifications	Observer list on Gateway; fires on PENDING and on the terminal status
Out of scope	Refunds, persistence, fraud checks	Not designed
Flagged, not built	Idempotency enforcement	idempKey is carried on Payment; production would check it before processing

Entities

Entity	Kind	Fields / Methods	Responsibility
PaymentMethod	Abstract class	validate(), type()	Owns its data and validation; names itself for routing
CardPayment	Subclass	cardNum, holder, exp, cvv	Card data and card validation
UpiPayment	Subclass	upiId	UPI data and UPI validation
Payment	Data holder	payId, orderId, idempKey, method, amount, status	Plain record of one payment attempt
PaymentStatus	Enum	PENDING, SUCCESS, FAILED	Lifecycle state
PaymentType	Enum	CARD, UPI	Routing key

Components and Responsibilities

Class	Responsibility
Gateway	Orchestrator. Creates Payment, validates, routes by type(), sets status, notifies observers
PaymentProcessor (interface)	void pay(amount, method) throws PaymentException
CardProcessor, UpiProcessor	Call BankService for their method type. No retry logic
RetryProcessor	Decorator around any processor. Retries only on RetryableException
BankService	Shared external call. Succeeds or throws
PaymentObserver	onStatusChanged(Payment, PaymentStatus); implemented by CustomerNotifier and MerchantNotifier

Main Flow

1. Merchant calls **gateway.pay(orderId, idempKey, amount, method)**.
2. Gateway creates Payment (PENDING) and notifies observers.
3. **method.validate()**. A failure goes straight to FAILED and is never retried.
4. **processors.get(method.type())** returns a RetryProcessor, which calls the real processor, which calls the bank.
5. Success sets SUCCESS; an exhausted or non-retryable failure sets FAILED.
6. Gateway notifies observers of the terminal status and returns Payment.

Design Decisions

Decision	Pattern	Why
Map keyed by method.type()	Strategy, OCP	No instanceof chain. Adding PayPal is a new method class, a new processor and one put; Gateway is untouched
RetryProcessor wraps any processor	Decorator, SRP	Retry lives in one place. Retry count is set at wiring time (card: 1, UPI: 3)
Exception split	Failure handling	ValidationException and NonRetryableException fail immediately; only RetryableException is retried
Observer list on Gateway	Observer, SRP	Gateway drives every status change so it knows when to publish; Payment stays a plain data holder
Per-observer try/catch	Isolation	One failing notifier cannot affect the payment result or the other observers
One downcast per processor	Accepted trade-off	Safe because Gateway routes by type(); generics or Visitor would remove it but add complexity

Known Limitations (say these before being asked)

Limitation	Production fix
Retries are synchronous and block the caller	Async queue with backoff
Idempotency is not enforced; a timeout after a successful charge could double charge on retry	Check idempKey before processing; pass keys to the bank
Payment is not thread-safe	Synchronize status updates or use compare-and-set
Observers run synchronously in the payment path	Publish to an event queue
Observers get the mutable Payment and may read a stale transition	Pass status as a parameter, or use an immutable event